RELATÓRIO TÉCNICO DE PENDÊNCIAS E SOLUÇÕES - ERP JB TINTAS

⚙️ PARTE 1: BACK-END E REGRAS DE NEGÓCIO (Python / Django)

Foco principal: Arquitetura "Fat Models, Utility Services, Thin Views", garantindo a inteligência oculta do sistema no servidor.

1.1. Leitura de Arquivo XML da NF-e com Bloqueio de Duplicidade

- **A Pendência:** O sistema precisa permitir que o gerente faça o upload do XML do fornecedor para alimentar o estoque, barrando notas enviadas duas vezes.
- **Onde Prometemos:** Tabela 2 do Relatório Parcial ("Planejado").
- **A Solução Técnica:**
    - **Como fazer:** Criar uma função no `services.py` que receba o arquivo `.xml`.
    - **Lógica:** Usar a biblioteca nativa do Python (`xml.etree.ElementTree` ou `xmltodict`) para ler as tags do arquivo.
    - **Validação:** Antes de ler os itens, o sistema pega a `<chNFe>` (Chave de Acesso) e faz um `EntradaXML.objects.filter(chave_acesso=chave)`. Se existir, retorna erro ("Nota já lançada"). Se não, cadastra a nota e faz um loop pelos produtos somando as quantidades na tabela `Produto`.

1.2. Cálculo de Comissões e Programa de Pontos de Fidelidade

- **A Pendência:** A cada venda, o vendedor deve receber sua comissão e o cliente deve acumular pontos (R$ 1,00 = 20 pontos).
- **Onde Prometemos:** Seções 2.2 e 2.5 do Relatório Parcial.
- **A Solução Técnica:**
    - **Como fazer:** Tudo isso deve ocorrer dentro de um bloco `@transaction.atomic` no `services.py` durante a finalização da venda.
    - **Lógica:** Após gravar a Venda e deduzir o estoque, o sistema multiplica o `valor_total` pelo percentual de comissão do `Usuario` (vendedor) e salva. Em seguida, pega o `valor_total`, multiplica por 20 e soma no campo `pontos` do `Cliente` vinculado à venda.

🖥️ PARTE 2: FRONT-END E INTERATIVIDADE (HTML / JavaScript)

Foco principal: Melhorar a experiência no balcão da loja e cumprir a exigência da Univesp de "Uso de Scripts Web e APIs".

2.1. Integração com a API do ViaCEP

- **A Pendência:** O cadastro de clientes exige digitação manual de endereço, o que gera lentidão.
- **Onde Prometemos:** Objetivos Específicos (Seção 2.1 do Relatório Parcial).
- **A Solução Técnica:**
    - **Como fazer:** Na tela `cadastro.html`, no campo de CEP, adicionar um ouvinte de evento JavaScript (`addEventListener('blur')` ou `keyup` quando bater 8 dígitos).
    - **Lógica:** Usar a função `fetch('[https://viacep.com.br/ws/](https://viacep.com.br/ws/){cep}/json/')`. Ao receber a resposta, o JS preenche automaticamente os campos `rua`, `bairro` e `cidade` na tela, deixando o foco (cursor) direto no campo "Número".

2.2. PDV Inteligente: Bipe, Troco e Validação em Tempo Real

- **A Pendência:** O caixa precisa ler código de barras, calcular o troco para evitar erros de saldo e validar CPFs antes de fechar a compra.
- **Onde Prometemos:** Requisitos mapeados pela comunidade (Seção 2.4.2 do Relatório Parcial).
- **A Solução Técnica:**
    - **Bipe:** No `pdv.html`, um input de "Código de Barras" fica sempre em foco (atributo `autofocus`). Quando o leitor "bipa", ele envia um *Enter* automático. O JS intercepta esse Enter, busca o produto no banco (via requisição assíncrona) e adiciona na tabela da tela.
    - **Troco:** Um campo "Valor Recebido" aparece se a opção for "Dinheiro". O JS subtrai o Valor Total do Valor Recebido e exibe o "Troco" em verde. Se for negativo, bloqueia o botão de finalizar.
    - **Validação:** Se o CPF digitado não tiver 11 dígitos válidos, a borda do input fica vermelha e bloqueia o botão.

2.3. Dashboard de Indicadores Visuais

- **A Pendência:** O dono da loja precisa ver de forma gráfica o faturamento diário e os produtos mais vendidos.
- **Onde Prometemos:** Sugestão direta da loja (Feedback da Quinzena 4).
- **A Solução Técnica:**
    - **Como fazer:** No arquivo `index.html` (tela do gerente), importar a biblioteca gratuita **Chart.js**.
    - **Lógica:** O Django manda os totais agrupados para a tela. O JavaScript cria um Gráfico de Barras (para faturamento dos últimos 7 dias) e um Gráfico de Pizza (para os 5 produtos mais vendidos).

🔒 PARTE 3: CONTROLE E SEGURANÇA (Gestão)

Foco principal: Transformar o sistema em um ERP profissional, resolvendo a dor de descontrole gerencial da JB Tintas.

3.1. Abertura e Fechamento de Caixa

- **A Pendência:** Evitar furos financeiros controlando o dinheiro que entra e sai da gaveta física.
- **Onde Prometemos:** Introdução do Relatório Parcial (dores levantadas na loja).
- **A Solução Técnica:**
    - **Como fazer:** Criar uma tabela `CaixaDiario` no `models.py` com os campos: `data`, `operador`, `fundo_inicial` (troco inicial da manhã), `total_vendas`, `saldo_final` e `status` (Aberto/Fechado).
    - **Lógica:** O vendedor não consegue abrir a tela do PDV se não clicar em "Abrir Caixa" e informar as notas e moedas iniciais. À noite, ele clica em "Fechar Caixa".

3.2. Controle de Acesso (RBAC) e Trilha de Auditoria

- **A Pendência:** Vendedores não podem acessar relatórios e o gerente precisa saber quem alterou o sistema.
- **Onde Prometemos:** Organização de segurança para viabilidade prática.
- **A Solução Técnica:**
    - **Acesso:** Usar os decorators do Django (`@login_required` e `@user_passes_test`). A rota de `relatorios.html` e `entrada_carga.html` é bloqueada se o usuário logado for `perfil == 'Vendedor'`.
    - **Auditoria:** Criar um modelo `LogAuditoria`. Sempre que uma venda for cancelada no `services.py`, salva-se no banco: *"Venda X cancelada pelo usuário Y às 14:30"*.

🚀 PARTE 4: TESTES E IMPLANTAÇÃO (QA e DevOps)

Foco principal: Garantir as notas obrigatórias da Quinzena 5 (Testes) e Quinzena 7 (Deploy e Usabilidade).

4.1. Testes Unitários Automatizados (`tests.py`)

- **A Pendência:** A aplicação não possui testes programados para provar que a lógica não falha.
- **Onde Prometemos:** Exigência direta da orientação da Quinzena 5.
- **A Solução Técnica:**
    - **Como fazer:** No arquivo `inventario/tests.py`, importar a classe `TestCase` do Django.

- **Lógica do Código:**
    1. *Teste 1:* Criar um produto com estoque = 5. Rodar a função de venda pedindo 10. O assert deve garantir que o sistema retorna Erro (Estoque negativo evitado).
    2. *Teste 2:* Enviar um XML com nota duplicada e garantir que a função bloqueia.

4.2. Hospedagem na Nuvem (Deploy Final)

- **A Pendência:** O sistema precisa rodar na internet e não apenas no computador local.
- **Onde Prometemos:** Tema Norteador (Requisito Nuvem) e Tabela 2.
- **A Solução Técnica:**
    - **Como fazer:** Usar a conta no **PythonAnywhere**. Fazer o pull (baixar) do código mais atual do GitHub.
    - **Lógica:** Configurar a aba "Web", rodar `python manage.py makemigrations` e `migrate` no console para criar o banco SQLite na nuvem. Rodar `python manage.py collectstatic` para o CSS e JavaScript não quebrarem.

4.3. Testes de Uso Prático na Loja e Acessibilidade

- **A Pendência:** Cronometrar a agilidade do sistema no balcão real e gerar laudos de acessibilidade.
- **Onde Prometemos:** Seção 2.4.3 e 2.5.6 do Relatório Parcial.
- **A Solução Técnica:**
    - **Como fazer:** Com o site online no PythonAnywhere, abrir o sistema na JB Tintas. Cronometrar o tempo de realizar 3 vendas. Pedir a opinião do gerente.
    - **Lighthouse:** Apertar F12 no Chrome e ir na aba "Lighthouse". Rodar a auditoria de Acessibilidade e tirar prints da nota para colar no Relatório Final (provando o cumprimento da WCAG).